

DemoBlaze Automation Framework – Plan / Design Document

1. Project Overview

This project is an end-to-end automation testing framework developed for the DemoBlaze web application. The framework is built using Selenium WebDriver with Java and follows the Page Object Model (POM) design pattern to ensure scalability, maintainability, and reusability.

The objective of this project is to automate core user flows such as authentication, product browsing, cart operations, order placement, and form validations.

2. Objectives

- Automate critical user journeys of the DemoBlaze application
- Ensure modular and maintainable test architecture
- Implement data-driven testing using JSON and DataProvider
- Integrate reporting using Extent Reports
- Handle dynamic elements and alerts effectively
- Ensure test independence using dynamic test data (unique users)

3. Tech Stack

- Programming Language: Java
- Automation Tool: Selenium WebDriver
- Test Framework: TestNG
- Build Tool: Maven
- Design Pattern: Page Object Model (POM)
- Reporting: Extent Reports
- Data Handling: JSON (Gson)
- Utilities: WebDriverManager, Apache Commons IO

4. Framework Architecture

The framework follows a layered architecture:

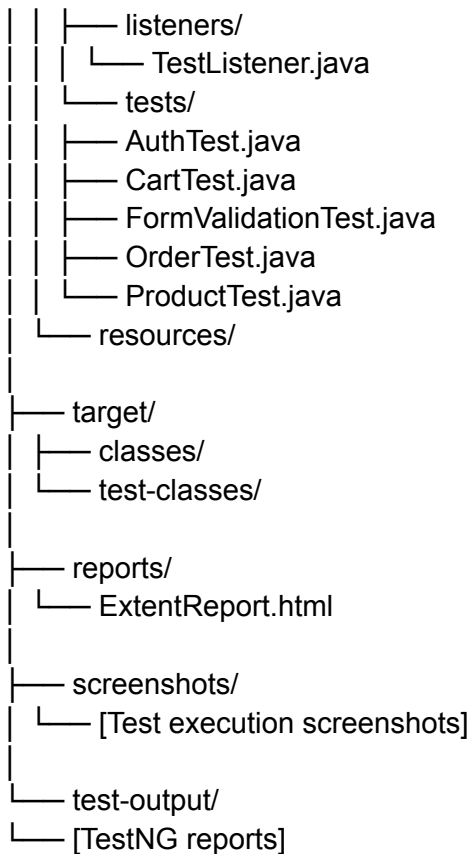
- Test Layer: Contains all test classes
- Page Layer: Contains page objects with UI interactions

- Base Layer: Common setup and teardown logic
- Utility Layer: Reusable utilities like config, screenshots, JSON, reporting
- Data Layer: External test data (JSON)

This separation ensures low coupling and high cohesion.

5. Project Structure

```
demoblaze-automation/
|
|— pom.xml
|— testng.xml
|— README.md
|
|— src/
|   |— main/
|   |   |— java/
|   |   |   |— com/srm/hackathon/demoblaze/
|   |   |   |   |— base/
|   |   |   |   |   |— BasePage.java
|   |   |   |   |   |— factory/
|   |   |   |   |   |   |— DriverFactory.java
|   |   |   |   |   |   |— DriverManager.java
|   |   |   |   |   |— pages/
|   |   |   |   |   |   |— CartPage.java
|   |   |   |   |   |   |— HomePage.java
|   |   |   |   |   |   |— LoginModalPage.java
|   |   |   |   |   |   |— OrderPage.java
|   |   |   |   |   |   |— ProductDetailPage.java
|   |   |   |   |   |   |— ProductPage.java
|   |   |   |   |   |— utils/
|   |   |   |   |   |   |— ConfigReader.java
|   |   |   |   |   |   |— ExtentManager.java
|   |   |   |   |   |   |— JsonUtil.java
|   |   |   |   |   |   |— ScreenshotUtils.java
|   |   |   |   |   |   |— WaitUtils.java
|   |   |— test/
|   |   |   |— java/
|   |   |   |   |— com/srm/hackathon/demoblaze/
|   |   |   |   |   |— base/
|   |   |   |   |   |   |— BaseTest.java
|   |   |   |   |   |— dataprovider/
|   |   |   |   |   |   |— DataProviders.java
```



6. Design Approach

Page Object Model (POM)

Each page is represented as a class containing locators and methods. This ensures reusability and separation of UI logic from test logic.

Base Classes

- BaseTest: Handles WebDriver initialization and teardown
- BasePage: Provides reusable methods like click, sendKeys, waits

Utilities

- ConfigReader: Reads configuration values
- ScreenshotUtils: Captures screenshots for failures
- JsonUtil: Reads test data from JSON files
- ExtentManager: Manages report generation

7. Test Modules

7.1 Authentication Module

- User Registration
- Login (valid/invalid)
- Logout

7.2 Product Module

- Navigate categories (Phones, Laptops, Monitors)
- Select products dynamically
- Validate product details

7.3 Cart Module

- Add product to cart
- Add multiple products
- Delete product from cart
- Validate total price

7.4 Order Module

- Place order with valid details
- Verify order confirmation
- Validate order ID presence
- Validate behavior for empty fields

7.5 Form Validation Module

- Signup with existing user
- Login with empty credentials
- Verify modal field value retention

8. Data-Driven Testing

Test data is stored in JSON format and accessed using DataProvider. This enables reusability and scalability of test cases without modifying code.

Example:

- orderData.json contains user details for order placement

9. Reporting

Extent Reports is integrated to generate a single consolidated HTML report for each execution.

Features:

- Test status (Pass/Fail)
- Execution time
- Screenshots on failure
- System information

Report Location:

reports/ExtentReport.html

10. Error Handling

- Explicit waits used for synchronization
- Alert handling for JavaScript popups
- Try-catch blocks for unstable UI behavior
- Logging for debugging

11. Parallel Execution

TestNG is configured to run tests in parallel at the test level. This improves execution speed while maintaining test independence.

12. Assumptions

- Application behavior may be inconsistent for some validations
- Alerts are native JavaScript alerts and cannot be captured in screenshots
- Each test uses a unique user to avoid data conflicts

13. Limitations

- No API or backend validation
- No cross-browser execution implemented
- Limited negative testing due to application constraints

14. Future Enhancements

- Cross-browser testing (Chrome, Firefox, Edge)

- CI/CD integration (Jenkins)
- Docker-based execution
- API automation integration
- Retry mechanism for flaky tests
- Advanced reporting with history tracking

15. Conclusion

This automation framework provides a scalable and maintainable solution for testing the DemoBlaze application. It follows industry best practices such as POM, data-driven testing, and structured reporting, making it suitable for real-world applications and extensible for future needs.